

How to Find It

A Discovery Manual for Competitive Programming

Competitive Programming Course

2026-08-01

Table of contents

How to use this document	6
Section 1 — Complexity, I/O, and constraints	8
Original problem: Packet checksum audit	8
Section 2 — Simulation and brute force	9
Original problem: Clockwork display	9
Section 3 — Counting and frequencies	10
Original problem: Balanced sticker box	10
Section 4 — Prefix and difference arrays	11
Original problem: Festival brightness budget	11
Section 5 — Sorting as a tool	12
Original problem: Minimum calibration spread	12
Section 6 — Two pointers and sliding windows	13
Original problem: Alarm-rich interval	13
Section 7 — Foundation mixed contest	14
Original problem: Noisy checkpoint log	14
Section 8 — Binary search on the answer	15
Original problem: Drone relay radius	15
Section 9 — Greedy exchange	16
Original problem: Single-use translation coupons	16
Section 10 — Constructive greedy	17
Original problem: Lexicographically smallest altitude plan	17
Section 11 — Intervals and sweep line	18
Original problem: Weighted radio congestion	18
Section 12 — Stacks, queues, and deques	19
Original problem: Roofline receiver count	19
Section 13 — Offline processing	20
Original problem: Historical rank queries	20
Section 14 — Core patterns mixed contest	21
Original problem: Quiet-zone threshold	21
Section 15 — BFS and unweighted shortest paths	22
Original problem: Combination dial escape	22

Section 16 — DFS, components, and cycles	23
Original problem: Mechanical linkage audit	23
Section 17 — Topological sorting and DAG DP	24
Original problem: Laboratory activation	24
Section 18 — DSU and minimum spanning trees	25
Original problem: Prewired research campus	25
Section 19 — Dijkstra and weighted modeling	26
Original problem: Toll-on-entry delivery	26
Section 20 — 0-1 BFS	27
Original problem: Mosaic repaint route	27
Section 21 — Graphs I mixed contest	28
Original problem: Emergency gates	28
Section 22 — One-dimensional DP	29
Original problem: Uneven paving	29
Section 23 — Knapsack and subset DP	30
Original problem: Expedition crate	30
Section 24 — Grid DP	31
Original problem: Turn-limited courier	31
Section 25 — Interval DP	32
Original problem: Symmetric deletion printer	32
Section 26 — Tree DP basics	33
Original problem: Relay-independent villages	33
Section 27 — Bitmask DP	34
Original problem: Tool-dependent laboratory tour	34
Section 28 — DP I mixed contest	35
Original problem: Energy-coded message	35
Section 29 — Modular arithmetic	36
Original problem: Repeated checksum block	36
Section 30 — GCD, LCM, and Diophantine equations	37
Original problem: Offset maintenance windows	37
Section 31 — Sieve, primes, and factorization	38
Original problem: Divisor-growth stream	38
Section 32 — Combinatorics under modulo	39
Original problem: Committee with separated seniors	39
Section 33 — Inclusion-exclusion	40
Original problem: Multiservice accounts	40

Section 34 — Probability and expected value	41
Original problem: Elevator with a reset floor	41
Section 35 — Math mixed contest	42
Original problem: Primitive diagonal tiles	42
Section 36 — Fenwick trees	43
Original problem: Streaming dominance count	43
Section 37 — Segment trees	44
Original problem: Alternating runway lights	44
Section 38 — Static queries and binary lifting	45
Original problem: Ancestral tax jumps	45
Section 39 — Heaps and priority queues	46
Original problem: Expiring top scores	46
Section 40 — Data structures mixed contest	47
Original problem: Warehouse command stream	47
Section 41 — Euler tour, LCA, and path lifting	48
Original problem: Three-courier meeting village	48
Section 42 — Advanced data structures mixed	49
Original problem: Versioned temperature ranks	49
Section 43 — String algorithms I	50
Original problem: Repeating header monitor	50
Section 44 — String algorithms II	51
Original problem: Shortest distinguishing command	51
Section 45 — Flows and matchings	52
Original problem: Volunteer shift allocation	52
Section 46 — Rerooting and advanced tree DP	53
Original problem: Squared-distance broadcast cost	53
Section 47 — Meet in the middle	54
Original problem: Balanced sign assignment	54
Section 48 — DP optimization	55
Original problem: Bounded-delay signal	55
Section 49 — Convex hull trick and Li Chao trees	56
Original problem: Quadratic launch calibration	56
Section 50 — Advanced algorithms mixed contest	57
Original problem: Archive similarity routes	57
Section 51 — Invariants in hard problems	58
Original problem: Paired counter increments	58

Section 52 — Game theory	59
Original problem: Mirror-token race	59
Section 53 — Advanced combinatorics	60
Original problem: Necklaces with no adjacent black beads	60
Section 54 — Geometry	61
Original problem: Fence gate crossing	61
Section 55 — Randomization and hashing	62
Original problem: Mirrored log updates	62
Section 56 — Hard greedy and matroids	63
Original problem: Maximum-value acyclic sensor links	63
Section 57 — Reductions and modeling	64
Original problem: Dual-channel observatory	64
Section 58 — Full virtual-contest workflow	65
Original problem set: The four-terminal mock	65
Section 59 — ICPC team strategy	66
Original simulation: Harbor-control board	66
Section 60 — Final capstone	67
Original problem: Flooded archive evacuation	67
Section 61 — ICPC contest readiness	68
Original drill: Reservation allocator	68
Section 62 — Codeforces course ladder	69
Original ladder problem: One-free prefix budget	69
Section 63 — AtCoder difficulty ladder	70
Original ladder problem: Painted staircase coupons	70
A reusable discovery checklist	71

How to use this document

This manual is about discovering algorithms, not recognizing answers after they have been revealed. Every worked problem here is original and does not appear among the course exercises. The examples therefore train the same concepts without spoiling the rest of the course.

The narratives are pedagogical reconstructions, not transcripts of private hidden reasoning. They retain the useful parts of discovery: a correct slow model, failed hypotheses, counterexamples, the observation that compresses the problem, and the proof obligation that remains.

For each problem, pause before reading **The turn** and write:

```
What is the smallest obviously correct algorithm?
Which dimension makes it too slow?
What information can two partial solutions forget safely?
What tempting rule can I break with five or fewer objects?
What invariant would make one transition undeniable?
```

The recurring discovery loop is:

```
exact model -> bottleneck -> structure -> compressed model -> proof -> oracle
```

Here is what each stage means.

1. Exact model: start with something obviously correct. Ignore efficiency for a moment. Enumerate every subset, keep the complete history of a walk, try every possible next choice, or write the direct recurrence. The exact model is your reference point: it states what the problem really asks without relying on an optimization that has not yet been justified.

For example, in a path problem with one discount coupon, an exact state might store the current vertex and the entire path taken so far. This contains enough information, but far more than the future needs.

2. Bottleneck: identify precisely why the exact model is too large. Do not stop at “it is slow.” Name the dimension responsible for the explosion: 2^n subsets, $n!$ orders, all path histories, an extra $O(n)$ scan inside every transition, or a state table whose dimensions multiply beyond the memory limit. Compare the resulting operation and memory counts with the constraints.

3. Structure: look for equivalence or order inside that dimension. Ask which differences between two partial solutions can still affect the future. Typical answers are:

- only an aggregate matters, such as a sum, minimum, remainder, or frequency;
- choices become monotone after sorting;
- several histories have the same current position and remaining resources;
- an exchange can turn one feasible solution into another;
- a graph, interval, or algebraic interpretation exposes a known operation.

This observation is the reason compression may be possible; the name of an algorithm usually comes afterward.

4. Compressed model: retain only future-relevant information. A compressed model merges exact situations that the rest of the problem cannot distinguish. It is not data compression in the file-storage sense. It is a smaller state space or representation that forgets irrelevant history.

In the coupon path example, suppose every future move depends only on the current vertex and whether the coupon has already been used. Then all paths ending in the same pair can be represented by

```
(current vertex, coupon_used)
```

instead of

```
(current vertex, complete path history)
```

The best known distance for that pair is sufficient; the identities and order of earlier vertices no longer matter. Similarly, subset-sum enumeration merges all subsets producing the same sum into one state, “this sum is reachable.”

The safety question for every proposed compression is:

If two exact situations become the same compressed state, can every possible future continuation treat them identically?

If the answer is no, the model has forgotten necessary information. For example, storing only the current vertex would be unsafe in the coupon problem: arriving with the coupon available is different from arriving after using it.

5. Proof: justify both the model and the algorithm. First prove that the compressed state preserves everything relevant to future choices. Then prove that every transition corresponds to a legal action and that every legal solution can be represented by transitions. Finally justify the algorithm used on that model—such as the greedy exchange, DP recurrence, or shortest-path property. Complexity analysis comes after the state and transitions are clear.

6. Oracle: compare against independent truth on small cases. Implement the exact model for tiny inputs and compare its answers with the optimized model. Random generation is useful, but deliberately include boundaries and cases that differ only in information you discarded. The oracle does not prove the algorithm; it helps expose a false compression, missing transition, or indexing mistake before submission.

When reading a worked problem, pause at every arrow. Try to write the artifact for that stage: the brute-force state, its operation count, the candidate equivalence, the compressed state meaning, one proof obligation, and one small oracle. Reading only the final algorithm trains recognition; reconstructing these intermediate artifacts trains discovery.

Section 1 — Complexity, I/O, and constraints

Original problem: Packet checksum audit

Problem. There are many packets. A packet contains a declared checksum and then n signed readings. Report whether the checksum equals the sum of its readings and whether any adjacent readings differ by more than a limit L . The sum of all packet lengths is at most two million.

Start with correctness. The statement asks for two properties of each packet. One scan can maintain the running sum and the previous reading. There is no reason to store all packets or compare every pair of readings.

Tempting mistake. Reading until end-of-file without respecting each n can silently consume the next packet when $n = 0$. A second mistake is calling the algorithm $O(\text{number of packets} * \text{maximum } n)$. The input contains only the actual readings, so the meaningful bound is the total number of readings.

The turn. Let N be the sum of all packet lengths. Every token is read a constant number of times, so time is $O(N)$ and auxiliary memory is $O(1)$ per packet. The checksum may require 64-bit storage even if each reading fits in 32 bits.

Transfer. Before inventing an algorithm, convert the constraints into a budget: how many loop iterations, stored objects, and accumulated units can actually occur across all test cases?

Section 2 — Simulation and brute force

Original problem: Clockwork display

Problem. A circular display has eight cells. Commands rotate it, reverse a chosen arc, or swap two cells. The initial display is unknown but is one of at most 2^8 binary patterns. Count how many initial patterns finish with exactly three lit cells in specified positions.

Slow but exact model. Enumerate all 256 initial patterns and simulate every command literally. This is not a fallback; the constraints are an invitation to exhaustive search.

Tempting mistake. Trying to reason about each final cell independently fails because reversals and swaps move dependencies between cells. Conversely, copying the entire display after every command is correct but unnecessary; an eight-element array is already tiny.

The turn. Bound the work before optimizing: $256 * \text{commands} * 8$ operations is easily safe. Encode a pattern by a bitmask, decode it into a small array, apply exactly the statement's operations, and test the final property.

Proof and oracle. Every allowed initial state appears once in the mask loop, simulation is deterministic, and the final predicate is checked exactly; therefore the count is exact. For later optimized solutions, this same program is a valuable random-test oracle.

Transfer. Brute force is a designed algorithm when the full search space, not just one dimension, fits the operation budget.

Section 3 — Counting and frequencies

Original problem: Balanced sticker box

Problem. A box contains colored stickers. In one operation, recolor any sticker. Find the minimum operations needed so that every color that remains appears between `low` and `high` times; colors may disappear completely.

First model. Positions do not matter, only counts. Replace the string or array by a frequency table. For each possible set of surviving colors, excess stickers can supply deficits.

Tempting mistake. Treating every color independently by summing $\max(0, \text{count} - \text{high})$ and $\max(0, \text{low} - \text{count})$ double-counts operations: one recolored excess sticker can fill one deficit. It also forgets that deleting a rare color may be cheaper than raising it to `low`.

The turn. Sort the nonzero frequencies. Try each suffix as the set of surviving colors. Counts before the suffix are donated completely; counts in the suffix are clipped to `[low,high]`. Compare total supply with total demand, and count unmatched removals or additions through one conservation equation.

Invariant. Once the survivor threshold is fixed, keeping a smaller frequency while deleting a larger one is never better: exchanging their roles does not increase either deficit or excess. This justifies suffix candidates.

Transfer. When labels are interchangeable and order is irrelevant, replace objects by counts first; then ask whether conservation couples what initially look like independent corrections.

Section 4 — Prefix and difference arrays

Original problem: Festival brightness budget

Problem. There are n lamps. Each of q operations adds brightness v to every lamp in $[l, r]$. After all operations, answer the total brightness in many queried intervals and report the first lamp exceeding a safety limit.

Correct slow model. Apply every update to every covered lamp, then answer queries by summing. This exposes two repeated range operations and costs up to $O(nq)$.

Tempting mistake. A prefix sum helps the final queries but does nothing to accelerate the range updates if rebuilt after each operation.

The turn. Record only where each update starts and stops:

```
difference[l]    += v
difference[r + 1] -= v
```

One prefix scan reconstructs every lamp. A second prefix sum answers final interval totals. The first safety violation is found during reconstruction.

Invariant. Before position i , the running sum of the difference array is exactly the sum of updates whose intervals contain i . The $r+1$ subtraction removes an update immediately after its final covered position.

Mistake detector. Test a one-element interval at the last index; it catches both inclusive/half-open confusion and an out-of-bounds $r+1$ write.

Transfer. Prefix sums answer static range aggregation; difference arrays batch range updates. Multiple phases may legitimately use both.

Section 5 — Sorting as a tool

Original problem: Minimum calibration spread

Problem. Choose exactly k devices from n calibration values. The cost is the maximum chosen value minus the minimum chosen value. Find the minimum cost.

Slow model. Enumerate all k -subsets and measure their extremes. The objective depends only on order, not original positions.

Tempting mistake. Pick the k values closest to the global average. For $0, 1, 2, 100, 101$ and $k=2$, the average points toward the middle while either $0, 1$ or $100, 101$ is optimal.

The turn. Sort the values. If an optimal set has minimum at sorted index i and maximum at j , selecting a value outside $[i, j]$ is impossible, while skipping interior values cannot reduce the same extremes. Thus some optimum is a contiguous sorted block of length k .

Scan every block and minimize $a[i+k-1] - a[i]$.

Proof habit. Do not say “sorting makes nearby values close.” State the exchange: replace a selected outside value by an unselected interior value; neither extreme becomes worse.

Transfer. Sorting is useful when original order is irrelevant and the objective depends on ranks, adjacency in value space, or extremes.

Section 6 — Two pointers and sliding windows

Original problem: Alarm-rich interval

Problem. A binary timeline marks alarm minutes. Find the shortest contiguous interval containing at least p alarms, or report impossibility.

Slow model. Try every left endpoint and extend right until p alarms have appeared. Restarting the scan makes the worst case quadratic.

Tempting mistake. Binary searching the interval length works, but each feasibility scan adds a logarithmic factor and obscures a stronger monotonicity: once a fixed left endpoint is valid, moving right farther stays valid.

The turn. Move **right** only forward, maintaining the alarm count. Whenever the window is valid, advance **left** while it remains valid, recording each minimal window ending at this **right**.

Invariant. Before **right** advances, every shorter left boundary for the current right has already been tested or removed, and both pointers have moved at most n times in total.

Counterexample check. With $p=0$, the empty interval question must be defined explicitly; constraints should usually require $p \geq 1$.

Transfer. A sliding window needs a property that can be maintained under adding the right item and removing the left item, plus a monotone direction for repairing validity.

Section 7 — Foundation mixed contest

Original problem: Noisy checkpoint log

Problem. Given timestamped checkpoint records, find the shortest time span containing every required checkpoint type. Timestamps may be unsorted and duplicate records are allowed.

Diagnosis before code. The words “shortest span” suggest a window, but the input order is not chronological. First sort by timestamp, then maintain frequencies of required types with two pointers. Sorting is a preprocessing reduction; the window is the core algorithm.

Tempting mistakes. A frequency table alone loses the span. A window on input order solves a different problem. Treating duplicate timestamps as a special case is unnecessary if the inclusive duration convention is stated.

Proof skeleton. For every right endpoint in chronological order, shrink to the latest left endpoint preserving all types. That is the shortest valid span ending at this right; taking the minimum over all right endpoints is global.

Contest-process lesson. Write two lines before choosing the tool:

```
order that has semantic meaning: timestamp order
property maintained inside candidate: required-type frequencies
```

Transfer. Mixed problems often require a short pipeline of familiar tools. Name each phase and what contract it provides to the next.

Section 8 — Binary search on the answer

Original problem: Drone relay radius

Problem. Villages lie on a line. Install at most k relay drones; one drone covers every village within distance R of its chosen position. Find the minimum integer R .

Correct slow model. Try every radius and determine how many drones are needed. Coordinates may be huge, so iterating radii is too slow.

The turn. For fixed R , sort villages and greedily cover the leftmost uncovered village by placing a drone R units to its right. It then covers as far right as possible. This yields a linear yes/no predicate.

If radius R works, every larger radius works. Binary search the first true radius.

Tempting mistake. Binary search is not justified merely because the answer is numeric. The predicate must be monotone. “Exactly k drones are used” is not the right predicate; “at most k suffice” is monotone.

Proof. Any solution must cover the leftmost uncovered village. Moving its responsible drone right until that village reaches the left edge cannot lose any later village, so the greedy placement is safe.

Transfer. Derive and prove `feasible(x)` first. Only then wrap it in binary search and state whether the boundary is first true or last true.

Section 9 — Greedy exchange

Original problem: Single-use translation coupons

Problem. Requests have required quality levels; coupons have quality levels. A coupon can serve one request if its quality is at least the request's requirement. Maximize served requests.

Slow model. This is a bipartite matching problem, but the compatibility edges have sorted threshold structure.

Tempting mistake. Give the strongest coupon to the hardest request first. It may work, but its proof is less direct and implementations often waste coupons when impossible requests appear. Giving arbitrary adequate coupons can also consume a strong coupon needed later.

The turn. Sort both lists. Process requirements from smallest upward and assign the smallest coupon that can satisfy the current request.

Exchange. If an optimum serves the smallest request with a stronger coupon while the greedy coupon serves a later request, swap the coupons; the stronger one can perform everything the weaker one did. If the weaker coupon was unused, the replacement is immediate.

Failure branch. A coupon smaller than the smallest remaining requirement can satisfy no future request and is safely discarded.

Transfer. In threshold matching, try to spend the least capable resource that completes the current obligation, then prove the exchange preserves all later options.

Section 10 — Constructive greedy

Original problem: Lexicographically smallest altitude plan

Problem. Construct a length- n sequence with values in $[0, H]$, starting at a , ending at b , and changing by exactly one each step. Among all valid sequences, output the lexicographically smallest or report impossibility.

Start from a completion test. At position i , a candidate next altitude x is completable exactly when the remaining steps can bridge $|x-b|$, with matching parity, while x respects the bounds.

Tempting mistake. Always step downward when possible. A locally smaller altitude may leave too much distance to the required endpoint. Checking only distance also misses parity: three ± 1 steps cannot change altitude by two.

The turn. Try $\text{current}-1$ before $\text{current}+1$; choose the first candidate passing the exact suffix test. If neither passes, construction is impossible.

Correctness. At the first position where two valid constructions differ, the algorithm takes the smaller value whenever it admits any completion. Therefore no valid construction can be lexicographically smaller.

Transfer. Lexicographic greedy is usually “try choices in order plus a feasibility oracle,” not “always take the smallest legal-looking choice.”

Section 11 — Intervals and sweep line

Original problem: Weighted radio congestion

Problem. Broadcast i contributes power w_i throughout half-open time interval $[\text{start}_i, \text{end}_i)$. Find the maximum total active power and the earliest time at which that maximum begins.

Slow model. Sampling every time unit fails when coordinates are large and misses the fact that the value changes only at endpoints.

The turn. Create events $+w$ at each start and $-w$ at each end, sort event coordinates, and combine all deltas at the same coordinate before evaluating the active total for the following interval.

Tempting mistake. Processing starts before ends one by one at equal times temporarily counts broadcasts that do not overlap under half-open semantics. Grouping equal coordinates removes this artificial state.

Invariant. After applying the combined delta at coordinate x , **active** equals the total power on $[x, \text{next}_x)$. Update the answer only for a nonempty following span, retaining the earliest coordinate on ties.

Transfer. A sweep line replaces a huge coordinate domain by the ordered moments at which the answer can change. Endpoint conventions determine event ordering or grouping.

Section 12 — Stacks, queues, and deques

Original problem: Roofline receiver count

Problem. Buildings stand left to right. Building i receives a signal from the first building to its right that is strictly taller; report that index or -1 .

Slow model. Scan right from every building, costing quadratic time on a decreasing skyline.

The turn. Scan from right to left with a stack of candidate indices. Before answering for height h , pop every candidate of height at most h : such a building cannot answer the current query, and the current building is closer and at least as tall for every building farther left.

The remaining stack top is the first taller receiver. Push the current index.

Tempting mistake. Pop only strictly smaller heights. Equal heights are not valid answers and would hide a later taller building. Storing heights without indices also loses the requested output.

Invariant. Stack indices increase away from the current scan position and their heights are strictly increasing from top to bottom. Every popped index is permanently dominated.

Transfer. A monotonic stack is discovered by asking which unresolved candidates a new element makes permanently irrelevant.

Section 13 — Offline processing

Original problem: Historical rank queries

Problem. Scores are inserted over time. Query (t, x) asks how many of the first t inserted scores are at least x . Queries may be answered after all input is read.

Slow model. Scan the first t scores for every query. A normal prefix sum cannot index arbitrary score thresholds.

The turn. Sort score insertions and queries by score descending. While processing threshold x , activate every score at least x at its time index. A Fenwick tree over time positions then returns the number activated in prefix $[1, t]$.

Tempting mistake. Sorting queries by t gives an easy active prefix but still requires a data structure for threshold counts; sorting by x makes the threshold condition monotone. Forgetting original query ids prints answers in processing order.

Invariant. Before answering threshold x , the Fenwick tree contains exactly positions whose scores satisfy `score >= x`.

Transfer. Offline processing searches for an order in which one query condition changes monotonically, allowing incremental updates rather than independent reconstruction.

Section 14 — Core patterns mixed contest

Original problem: Quiet-zone threshold

Problem. Given positive noise readings, find the minimum threshold X for which some contiguous interval of length at least k has total excess $\text{sum}(\max(0, a_i - X))$ at most budget B .

Diagnosis. The requested output is a minimum numeric threshold. For fixed X , increasing X can only decrease every excess, so feasibility is monotone. Inside $\text{feasible}(X)$, positive excesses support a sliding window: expand right, shrink while over budget, and test whether a valid window reaches length k .

Tempting mistakes. Binary searching without proving monotonicity is cargo cult. Using a window directly on raw readings answers a different constraint. Recomputing each window sum makes the predicate quadratic.

Proof contracts. The window invariant says its maintained sum equals the current interval's excess for fixed X . The binary-search invariant says all thresholds below low are impossible and high is feasible.

Transfer. In mixed problems, write a pipeline:

```
outer answer structure -> inner feasibility problem -> maintained invariant
```

Prove each arrow separately before combining implementations.

Section 15 — BFS and unweighted shortest paths

Original problem: Combination dial escape

Problem. A four-digit dial state can change one digit by $+1$ or -1 modulo ten. Some states are forbidden. Find the minimum moves from one state to another and reconstruct one shortest sequence.

Model first. Each legal dial configuration is a vertex; one allowed move is an edge. All edges cost one, so the number of moves is an unweighted shortest path.

Tempting mistake. Greedily move each digit toward its target. A forbidden state may require temporarily moving away. DFS can find a path but does not discover paths by increasing length.

The turn. BFS starts with distance zero and explores all states at distance d before any at $d+1$. Record a parent only on first discovery. When the target is reached, follow parents backward and reverse the sequence.

Invariant. When a state leaves the queue, its stored distance is minimum. Every possible shorter predecessor would have left earlier and discovered it.

Transfer. Translate “minimum number of operations” into a state graph; then inspect edge costs before choosing BFS, 0-1 BFS, or Dijkstra.

Section 16 — DFS, components, and cycles

Original problem: Mechanical linkage audit

Problem. An undirected graph describes joints and rods. For every connected component, classify it as a tree, a single-cycle mechanism, or a component with multiple independent cycles.

Exact component summary. DFS each unvisited vertex while counting vertices V_c and the sum of degrees. The edge count is half the degree sum.

Tempting mistake. Marking “cycle found” on any edge to a visited vertex counts the parent edge as a cycle. It also cannot distinguish one cycle from many.

The turn. For a connected undirected component:

```
E_c = V_c - 1 -> tree
E_c = V_c      -> exactly one independent cycle
E_c > V_c      -> multiple independent cycles
```

Why. A spanning tree uses V_c-1 edges. Every additional edge creates one independent cycle. Connectivity is supplied by the DFS itself.

Transfer. Component traversal often exists only to collect a sufficient summary. Decide which counts or flags classify the whole component before writing recursive logic.

Section 17 — Topological sorting and DAG DP

Original problem: Laboratory activation

Problem. Experiment v takes $\text{duration}[v]$ time and may start only after all prerequisite experiments finish. Unlimited laboratories are available. Find the earliest finish time of every experiment, or report a dependency cycle.

Model. Prerequisite edges form a directed graph. A node's answer depends on predecessors, so dependencies must be processed first.

Tempting mistake. Sorting by duration ignores prerequisites. Repeatedly scanning for currently available experiments can become quadratic.

The turn. Kahn's algorithm maintains indegrees. When processing edge $u \rightarrow v$, update:

```
start[v] = max(start[v], finish[u])
```

When v reaches indegree zero, all contributors are known and $\text{finish}[v] = \text{start}[v] + \text{duration}[v]$ is final.

Cycle certificate. If fewer than n vertices are removed, some dependency cycle prevents all remaining indegrees from becoming zero.

Transfer. DAG DP is ordinary dependency evaluation after finding an order that makes every transition point forward.

Section 18 — DSU and minimum spanning trees

Original problem: Prewired research campus

Problem. Some buildings are already connected free of charge. Candidate cables have costs. Find the minimum additional cost making every building communicate.

Reduction. Free connections and bought cables both create undirected connectivity. Initialize a DSU by uniting all free pairs, then process candidate cables by increasing cost.

Tempting mistake. Run a normal MST and merely assign free edges cost zero without actually including all free links in the edge list; omitted prewiring then disappears from the model. Prim from one building is valid but needs an adjacency representation for all candidates.

The turn. Kruskal buys an edge only when its endpoints currently belong to different DSU components. The cut property says the cheapest edge crossing any current component cut is safe.

Failure case. After all edges, more than one DSU component means connection is impossible; a cheap partial forest is not an answer.

Transfer. DSU represents an equivalence relation that only merges. Kruskal is appropriate when globally sorted edges and cycle rejection are sufficient.

Section 19 — Dijkstra and weighted modeling

Original problem: Toll-on-entry delivery

Problem. Directed roads have travel times, and entering city v charges a city toll. The starting city is free. Find the cheapest route to every city.

Modeling decision. Put the destination toll onto every incoming edge:

```
effective cost(u -> v) = road_time + toll[v]
```

Now route cost is an ordinary sum of nonnegative edge weights.

Tempting mistake. Add the toll when a city is popped from the priority queue. The same tentative path may then be charged inconsistently, and the starting city needs an exception. Encoding cost in edges makes relaxation local and unambiguous.

The turn. Dijkstra repeatedly finalizes the unsettled state with minimum tentative distance. Ignore stale heap entries whose value differs from the current distance.

Invariant. With nonnegative effective weights, any undiscovered path to the popped city must first pass through a vertex with distance no smaller, so it cannot improve the popped value.

Transfer. Before selecting a shortest-path algorithm, make the full state and the exact cost of one transition explicit.

Section 20 — 0-1 BFS

Original problem: Mosaic repaint route

Problem. Move through a character grid. Moving to a cell of the same color costs zero; moving to a different color costs one repaint token. Find the minimum tokens from start to goal.

Diagnosis. This is not unweighted BFS: two paths with equal step counts can have different token costs. It is Dijkstra with edge weights restricted to zero and one.

The turn. Maintain a deque. A successful zero-cost relaxation goes to the front; a one-cost relaxation goes to the back. The deque therefore keeps states in nondecreasing tentative distance without a heap.

Tempting mistake. Mark a cell permanently visited on first insertion. A later zero-cost route may improve it. Store distances and relax normally.

Invariant. Before removing the front, no queued vertex has smaller distance: zero edges preserve the current layer at the front, and one edges enter the next layer at the back.

Transfer. When transition costs have a tiny special alphabet, look for a specialized frontier structure before paying for a general priority queue.

Section 21 — Graphs I mixed contest

Original problem: Emergency gates

Problem. Roads are directed. Traversing forward is free, traversing a road backward consumes one override, and changing districts at designated portals has fixed positive cost. Find the minimum total cost.

Triage. The statement mixes graph features, so list transition weights before choosing a template. Reverse-road transitions cost one, forward cost zero, but portals may cost more than one. Therefore plain 0-1 BFS is invalid; the complete model requires Dijkstra.

Tempting mistake. Solve roads with 0-1 BFS and then add a portal adjustment. Portal use changes which road states become reachable, so the phases cannot be separated.

The turn. Build one state graph containing all road and portal transitions. If overrides had a limited count rather than a cost, expand state to `(vertex, overrides_used)`.

Transfer. In mixed graph problems, do not choose by story words. Inventory states, transitions, weights, and resource dimensions; the algorithm follows from that contract.

Section 22 — One-dimensional DP

Original problem: Uneven paving

Problem. A path of length n can be paved with slabs of lengths 1, 3, or 4. Slab type d has placement cost depending on its ending position. Find the minimum total cost.

Slow model. Recursively try every next slab. Different paving histories repeatedly reach the same covered prefix length.

The turn. Let $dp[i]$ be the minimum cost to cover exactly the first i units. The last slab has one of three lengths:

```
dp[i] = min(dp[i-d] + cost(d,i)) for d in {1,3,4}, d <= i
```

Tempting mistake. Greedily take the cheapest cost per unit. Position-based costs and leftover lengths break local ratios. Defining $dp[i]$ as “cost after considering i slab types” also ignores the natural progress coordinate.

Proof. Every optimal paving has one final slab; removing it leaves an optimal paving of the corresponding shorter prefix, otherwise substitution would improve the original.

Transfer. A good DP state identifies when two histories have identical future options. Here only the covered prefix length matters.

Section 23 — Knapsack and subset DP

Original problem: Expedition crate

Problem. Artifacts have mass, scientific value, and may each be selected once. Maximize value with total mass at most C , but require the crate to be nonempty.

Correct two-dimensional model. $dp[i][w]$ is the best value using the first i artifacts with exact mass w . Each artifact is skipped or taken once.

Compression. Row i depends only on row $i-1$, so keep one array. Iterate mass backward; forward iteration would reuse the same artifact through a value written earlier in the current iteration.

Tempting mistake. Value/mass ratio solves fractional knapsack, not 0/1 selection. Artifacts $(mass, value) = (2, 3), (3, 4), (4, 100)$ with capacity four make the distinction obvious.

Nonempty condition. Initialize unreachable exact masses to negative infinity rather than zero. Otherwise an impossible positive mass can look like a valid empty selection.

Transfer. Loop direction is part of the recurrence proof: backward for single use, forward for unlimited use.

Section 24 — Grid DP

Original problem: Turn-limited courier

Problem. Move right or down through a blocked grid. Count paths from the top-left to bottom-right that make at most K direction changes.

Why ordinary grid DP is insufficient. Two paths reaching the same cell may have different last directions and used-turn counts, which change future options. Position alone is not a sufficient state.

The turn. Use:

```
dp[row][col][turns][last_direction]
```

Extending in the same direction preserves **turns**; changing direction adds one. Process cells in row/column order because every transition comes from above or left.

Tempting mistake. Count all paths first and subtract paths with too many turns. The invalid property is global and overlapping; there is no simple subtraction without tracking it.

Proof. The state records exactly the information that affects legal next moves. Every path has a unique final transition into one state, so transitions neither omit nor double-count paths.

Transfer. Grid coordinates provide the DAG order; additional constraints decide which extra state dimensions are necessary.

Section 25 — Interval DP

Original problem: Symmetric deletion printer

Problem. Delete an entire string. Deleting one endpoint costs one. If the current two endpoints are equal, delete them together for cost one. Find the minimum cost.

State. Operations change both ends of a remaining contiguous interval, so let $\text{dp}[l][r]$ be the minimum cost to delete substring $l..r$.

Transitions. Delete the left endpoint alone, delete the right alone, or—if equal—delete both together after deleting the interior.

Tempting mistake. Greedily pair equal characters as soon as possible. Pairing outer equal characters can block a more valuable nested structure, and locally nearest equal partners need not be globally compatible.

Order. Compute increasing interval length so every smaller interior state already exists.

Proof. Classify an optimal solution by its first operation involving an endpoint. Each category corresponds to one transition, so minimizing them is complete.

Transfer. Interval DP appears when subproblems remain contiguous and the key decision splits an interval, removes an endpoint, or chooses its last operation.

Section 26 — Tree DP basics

Original problem: Relay-independent villages

Problem. Choose maximum-weight villages in a tree so that no chosen village is adjacent to another chosen village. Negative weights are allowed.

Root the tree. For each node u , future interaction with its parent depends only on whether u is chosen.

```
take[u] = weight[u] + sum(skip[child])
skip[u] = sum(max(take[child], skip[child]))
```

Tempting mistake. Repeatedly select the currently heaviest vertex and delete its neighbors. A moderately heavy parent can block several children whose combined weight is larger.

Proof. Once u 's chosen status is fixed, child subtrees are independent. If u is taken, each child must be skipped; otherwise each child chooses its better state.

Boundary. Allow choosing nothing, especially when all weights are negative.

Transfer. Rooting turns an acyclic undirected graph into parent-to-child dependencies. The DP state records exactly what the subtree exposes across its single boundary edge.

Section 27 — Bitmask DP

Original problem: Tool-dependent laboratory tour

Problem. There are at most 18 laboratories. Visiting lab i grants a tool; lab j may be visited only after all tools in its prerequisite mask are held. Travel cost depends on the last lab. Visit every lab at minimum cost.

State discovery. Future legality depends on the set of collected tools, not their acquisition order; travel cost also depends on current location.

```
dp[mask][last] = minimum cost to visit exactly mask and finish at last
```

Try every unvisited lab whose prerequisite mask is contained in `mask`.

Tempting mistake. Store only `mask`. Two tours with the same visited set but different last labs have different next travel costs. Store only `last`, and prerequisite legality is lost.

Proof. Every tour has a unique penultimate state obtained by removing its last visited lab. Conversely every legal transition appends one lab, so mask size gives an acyclic iteration order.

Transfer. Bitmask DP is appropriate when a small set controls future choices and identities matter more than counts.

Section 28 — DP I mixed contest

Original problem: Energy-coded message

Problem. Split a digit string into blocks of length one to three. A block is legal if its integer value is at most M ; choosing it adds a value-dependent energy. Minimize energy and count optimal splits.

Diagnosis. The input is a sequence, decisions partition a prefix, and a block looks back at most three positions. This is one-dimensional DP, not interval DP: after choosing the final block, only an earlier prefix remains.

Maintain for every prefix both minimum energy and number of ways achieving that minimum. For each legal final block, relax the pair: replace on smaller cost, add counts on equal cost.

Tempting mistake. A greedy longest legal block minimizes block count, not energy. Counting all legal splits and separately minimizing energy loses the coupling between the two outputs.

Transfer. In mixed DP, identify what remains after the last decision. A prefix suggests 1D DP, two free endpoints suggest interval DP, a small chosen set suggests bitmask DP, and independent child regions suggest tree DP.

Section 29 — Modular arithmetic

Original problem: Repeated checksum block

Problem. A decimal block B of length d is concatenated with itself k times. Compute the resulting integer modulo m , where k is enormous.

Algebraic model. Appending one block transforms checksum x into:

$$x \rightarrow x * 10^d + B \pmod{m}$$

This is an affine transformation. Repeating it k times can be done by binary exponentiation of transformations, composing (a, b) as $x \rightarrow ax+b$.

Tempting mistake. Use the geometric-series division formula. Division modulo m is invalid when 10^d-1 has no modular inverse. Transformation composition needs no division and works for composite moduli.

Invariant. After processing a bit prefix of k , the accumulated transform equals exactly that many repetitions; squaring a transform doubles its repeat count.

Transfer. When an operation repeats, search for an associative composition object—numbers, matrices, permutations, or affine maps—then exponentiate it.

Section 30 — GCD, LCM, and Diophantine equations

Original problem: Offset maintenance windows

Problem. Machine A is available at times $a + px$; machine B at $b + qy$, for nonnegative integers x, y . Find the earliest common time or report none.

Equation. A common time requires:

$$p \cdot x - q \cdot y = b - a$$

The linear Diophantine equation has an integer solution exactly when $\gcd(p, q)$ divides $b - a$.

Tempting mistake. Search multiples until they meet; periods can be near 10^{18} . Using only $\text{lcm}(p, q)$ handles zero offsets but not different starting times.

The turn. Extended Euclid constructs one solution. Scale it by $(b - a)/g$, then shift the solution family by multiples of q/g and p/g to make both counters nonnegative and minimize the resulting time.

Transfer. GCD gives existence; extended GCD gives a witness; parameterizing all witnesses handles bounds and optimization.

Section 31 — Sieve, primes, and factorization

Original problem: Divisor-growth stream

Problem. Values up to one million arrive. After each insertion, report the number of divisors of the product of all values so far, modulo a prime.

Factor viewpoint. If the running product has prime exponents e_p , its divisor count is $\text{product}(e_p+1)$. Precompute smallest prime factors once, then factor each arriving value in logarithmic time.

Tempting mistake. Multiply the values and factor the product; it overflows almost immediately. Re-sieving or trial-dividing each value ignores the shared upper bound.

Update. When exponent e for prime p increases by d , replace factor $e+1$ in the modular answer by $e+d+1$, using a modular inverse. Alternatively, recompute from exponent maps if the modulus contract does not permit inverses.

Invariant. The exponent table is the exact prime factorization of the conceptual product, although that product is never constructed.

Transfer. Products of many bounded values are often represented by aggregated prime exponents rather than by the integer itself.

Section 32 — Combinatorics under modulo

Original problem: Committee with separated seniors

Problem. Arrange J distinct juniors and S distinct seniors in a line so that no two seniors are adjacent. Count arrangements modulo a prime.

Construct a bijection. Arrange juniors first in $J!$ ways. They create $J+1$ gaps, including both ends. Choose S distinct gaps and permute seniors among them:

```
answer = J! * C(J+1,S) * S!
```

If $S > J+1$, the answer is zero.

Tempting mistake. Count all $(J+S)!$ arrangements and subtract those with an adjacent senior pair. Multiple adjacent pairs overlap, so simple subtraction double-counts; inclusion-exclusion would be much heavier.

Proof. Every valid arrangement has a unique junior order, unique chosen senior gaps, and unique senior order. The construction and decomposition are inverse mappings.

Transfer. Before writing factorials, find the combinatorial objects and a reversible construction that explains every factor.

Section 33 — Inclusion-exclusion

Original problem: Multiservice accounts

Problem. Count length- n strings over an alphabet containing ordinary letters plus special symbols @, #, and \$, with every special symbol used at least once.

Complement sets. Let A_s be strings missing special symbol s . We need the complement of $A_@ \cup A_\# \cup A_\$$.

Tempting mistake. Subtract the three missing-symbol counts and stop. Strings missing both @ and # were subtracted twice and must be added back.

The turn. For every subset of special symbols, forbid that subset and count strings over the remaining alphabet. Add terms for even forbidden-set size and subtract terms for odd size.

Why signs work. A string missing exactly m required symbols contributes the alternating binomial sum over subsets of those m symbols, which is zero unless $m=0$.

Transfer. Inclusion-exclusion is appropriate when violating several requirements is easy to count after choosing exactly which requirements are ignored.

Section 34 — Probability and expected value

Original problem: Elevator with a reset floor

Problem. From floor $i < N$, a button moves up one with probability p_i , but otherwise returns to floor zero. Find the expected presses to reach N .

First-step equations. Let $E[i]$ be expected remaining presses:

$$\begin{aligned} E[i] &= 1 + p_i E[i+1] + (1-p_i) E[0] \\ E[N] &= 0 \end{aligned}$$

Tempting mistake. Add expected waiting times $1/p_i$ for each floor. A failure at a high floor destroys earlier progress, so stages are not independent.

The turn. Express every $E[i]$ backward as $A[i] + B[i] * E[0]$. Substituting at $i=0$ gives one linear equation for $E[0]$. Detect the case whose denominator is zero, corresponding to infinite expectation.

Transfer. Expected-time recurrences often refer back to the start. Keep the self-reference symbolically rather than trying to compute states numerically before the equation closes.

Section 35 — Math mixed contest

Original problem: Primitive diagonal tiles

Problem. Count grid points (x, y) with $1 \leq x \leq A$, $1 \leq y \leq B$, and $\gcd(x, y) = 1$.

Small-model clue. Directly testing every pair is $O(AB)$. The property is defined by common divisors, suggesting inversion over divisibility counts.

The turn. For every d , count pairs where both coordinates are divisible by d : $\text{floor}(A/d) * \text{floor}(B/d)$. Weight this count by the Möbius function $\mu[d]$; the divisor sum of μ is one only for gcd one.

Tempting mistake. Subtract pairs divisible by each prime once. Pairs sharing multiple primes are over-subtracted—the exact situation inclusion-exclusion or Möbius inversion repairs.

Validation. Enumerate all pairs for tiny rectangles and compare. Also check $A=1$, where every $(1, y)$ is primitive.

Transfer. In math mixed problems, translate the predicate into a divisor, parity, or bijection statement before searching for formulas.

Section 36 — Fenwick trees

Original problem: Streaming dominance count

Problem. Values arrive one by one. After each arrival x , report how many previous values are strictly smaller than x .

Slow model. Scan the whole prefix for every arrival. We need dynamic prefix counts over value order.

The turn. Coordinate-compress all values. A Fenwick tree stores frequency at each rank. Before inserting rank r , query the prefix through $r-1$; then add one at r .

Tempting mistake. Query through r , which counts equal values despite the strict inequality. Compressing duplicates to different ranks causes the same error more subtly.

Invariant. Before processing position i , the tree contains exactly the frequency distribution of positions before i . Fenwick nodes store sums over power-of-two suffixes of prefixes, enabling update and prefix query in $O(\log n)$.

Transfer. Fenwick trees fit invertible prefix aggregation with point updates; compression maps large ordered coordinates to dense ranks.

Section 37 — Segment trees

Original problem: Alternating runway lights

Problem. A binary string receives range-flip updates. After each update, report the longest contiguous alternating segment in the whole string.

Why one number per segment fails. The best alternating segment may cross a node boundary. To merge children, store each node's left bit, right bit, longest alternating prefix, suffix, and internal best.

The turn. If the left child's right bit differs from the right child's left bit, their boundary is alternating: suffix and prefix can join. Otherwise no crossing segment exists.

Lazy flip insight. Flipping every bit preserves equality/difference between adjacent bits. Prefix, suffix, and best lengths remain unchanged; only boundary bit values toggle. This yields a constant-size lazy tag.

Tempting mistake. Store only the best length and expect to merge it. Two segments with equal best values can have different boundary compatibility.

Transfer. Design a segment-tree node by writing the exact formula for combining two adjacent answers; whatever the formula reads must be stored.

Section 38 — Static queries and binary lifting

Original problem: Ancestral tax jumps

Problem. A rooted forest gives each node a parent edge with tax. Queries ask for the ancestor reached after exactly k parent moves and the total tax paid, or report that the path leaves the forest.

Slow model. Follow k parents per query. Static parents allow preprocessing of powers of two.

Store for every node and bit j :

```
up[v][j]    = ancestor after  $2^j$  moves  
tax[v][j]   = tax over those moves
```

Combine two $2^{(j-1)}$ blocks to build level j . Decompose k by binary bits to answer a query.

Tempting mistake. Sum taxes after moving the node, which reads the wrong block. Accumulate the current node's block before replacing it by its ancestor.

Invariant. After processing bits below j , the current node and sum equal exactly that many requested moves.

Transfer. Binary lifting is a sparse table along a functional graph path, and aggregates can travel with the jump.

Section 39 — Heaps and priority queues

Original problem: Expiring top scores

Problem. Scores arrive with expiration times. At each timestamp, report the largest k unexpired scores and their sum.

Model. Time order activates and expires entries; rank order decides the top k . Use one min-heap for chosen top scores and one max-heap for the rest, plus expiration events and lazy deletion ids.

Tempting mistake. Keep only a max-heap and pop k values for every query; putting them back costs $O(k \log n)$ per timestamp. Removing arbitrary expired heap entries eagerly is also unsupported.

The turn. Maintain the invariant that the chosen heap contains exactly the k largest live scores. Rebalance sizes and swap boundary elements whenever the largest rest score exceeds the smallest chosen score. Lazy-delete expired ids whenever they reach a heap top.

Transfer. A heap manages one extreme. Maintaining a partition around a rank often needs two heaps plus a precise rebalancing invariant.

Section 40 — Data structures mixed contest

Original problem: Warehouse command stream

Problem. Commands insert labeled crates with priorities, cancel labels, increase all priorities in a label range, and ask for the current maximum.

Operation diagnosis. A heap supports maxima but not range updates or direct cancellation. A segment tree over compressed labels supports range addition and maximum, while a map tracks whether a label currently exists and its base value.

Tempting mistake. Choose the structure from the word “priority.” List every update and query first; one unsupported operation invalidates the template.

The turn. Compress every label that can appear. Segment leaves represent active priority or negative infinity, lazy tags represent range increments, and cancellation resets one leaf after accounting for pending tags.

Transfer. Mixed data-structure problems are interface-design tasks:

```
operation -> required primitive -> candidate structure -> missing primitive
```

Reject a candidate as soon as one required primitive lacks the target bound.

Section 41 — Euler tour, LCA, and path lifting

Original problem: Three-courier meeting village

Problem. Three couriers start at vertices a, b, c of a tree. Choose a village minimizing the sum of their travel distances.

Structural clue. Tree paths are unique. The optimal meeting village is the median: the common intersection point of the three pairwise paths.

Compute $\text{lca}(a, b)$, $\text{lca}(a, c)$, and $\text{lca}(b, c)$ under any fixed root; the deepest of these three is the median. Distances follow from depths and LCA.

Tempting mistake. Choose the LCA of all three. Its value depends on the arbitrary root and may lie above the true median. Testing every meeting vertex costs quadratic time across many queries.

Proof. Removing the median separates the three couriers into at most three branches. Moving away from it decreases distance to at most one courier while increasing distance to the other two, so total distance cannot improve.

Transfer. LCA is a primitive for path geometry; many query answers are small formulas involving several LCAs and depths.

Section 42 — Advanced data structures mixed

Original problem: Versioned temperature ranks

Problem. Point temperatures change over time. A query names an earlier version and asks for the number of values at most x in index interval $[1, r]$.

Diagnosis. The query has three dimensions: version, index range, and value threshold. A normal segment tree handles current versions only; a Fenwick tree handles one ordering at a time.

The turn. Build a persistent segment tree over compressed values for every prefix of updates within each version lineage, or—if versions are linear—use an offline time divide-and-conquer with a Fenwick tree. The statement’s version branching decides between them.

Tempting mistake. Say “persistent segment tree” before defining what one root represents. If roots represent time but queries also need index ranges, one dimension is still missing.

Transfer. For advanced structures, write the axes of every update and query. Choose which axis is ordered offline, which indexes the structure, and which is stored in node summaries.

Section 43 — String algorithms I

Original problem: Repeating header monitor

Problem. A stream is formed by appending characters. After each append, report the longest proper prefix of the current stream that is also a suffix and occurs at least twice without using the full string.

Slow model. Compare every prefix with the corresponding suffix after every append, leading to cubic behavior.

The turn. The prefix function stores, for each ending position, the length of the longest border. On a mismatch, the next candidate is not arbitrary: it is the longest border of the current candidate, already stored by the prefix function.

Tempting mistake. Reset candidate length to zero after every mismatch. This is correct but loses information and becomes quadratic on repetitive strings.

Invariant. Before comparing the new character, j is the longest prefix matching a suffix of the processed prefix. Following $\text{pi}[j-1]$ enumerates all smaller viable borders in decreasing order.

Transfer. KMP is discovered by asking what part of a failed match is known to match itself and can therefore be reused.

Section 44 — String algorithms II

Original problem: Shortest distinguishing command

Problem. Given many command words, print for each word its shortest prefix that is not a prefix of any other word, or IMPOSSIBLE if the word is a prefix of another command.

Model. Insert every word into a trie and store how many words pass through each node. For a word, the first node with pass count one ends its shortest distinguishing prefix.

Tempting mistake. Sort words and compare only with the previous word. A word's longest common prefix with the next neighbor may be larger. Sorting can still solve the problem, but both neighbors are required; the trie gives the property directly.

Proof. Before the first count-one node, at least one other word shares the prefix, so it is not distinguishing. At the first count-one node, only this word passes, so the prefix is valid and shortest.

Transfer. Tries turn shared prefixes into shared paths, allowing aggregate information to be stored exactly where prefixes diverge.

Section 45 — Flows and matchings

Original problem: Volunteer shift allocation

Problem. Volunteer i can work certain shifts and at most cap_i shifts. Shift j needs need_j volunteers. Determine whether all needs can be filled and construct an allocation.

Reduction. Build a flow network:

```
source -> volunteer i      capacity cap_i
volunteer i -> allowed j   capacity 1
shift j -> sink            capacity need_j
```

Full allocation exists exactly when maximum flow equals total shift demand.

Tempting mistake. Greedily fill the most constrained shift. A volunteer used there might be the only option for a different shift; local scarcity measures interact. Plain bipartite matching with unit source capacities also forgets volunteers may work several shifts.

Why the reduction is exact. Every integral unit of flow selects one allowed volunteer-shift pair. Capacities enforce both sides' limits. Conversely every valid allocation routes those units. Integer capacities guarantee an integral maximum flow.

Transfer. A good reduction proves both directions: solution-to-model and model-to-solution.

Section 46 — Rerooting and advanced tree DP

Original problem: Squared-distance broadcast cost

Problem. For every possible server vertex r , compute $\sum_v \text{distance}(r, v)^2$ in a tree.

First root. One DFS from root zero computes subtree sizes, sums of distances, and sums of squared distances.

The reroot turn. Move the root across edge $u-v$, where v 's subtree has size s . Distances inside that subtree decrease by one; all others increase by one. Expanding $(d-1)^2$ and $(d+1)^2$ shows that the new squared sum needs both the old squared sum and the sum of distances, not only subtree size.

Tempting mistake. Apply the standard linear-distance reroot formula to squares. Nonlinear objectives introduce additional moments.

Invariant. During the second DFS, stored global moments are correct for the current root; crossing one edge updates them by partitioning vertices into the child subtree and its complement.

Transfer. Derive reroot transitions algebraically by describing how every element's contribution changes across one edge.

Section 47 — Meet in the middle

Original problem: Balanced sign assignment

Problem. For up to 40 positive weights, assign each + or - to minimize the absolute signed sum.

Brute force. There are 2^{40} sign assignments, too many. But each half has at most 20 elements and about one million signed sums.

The turn. Enumerate all sums for both halves. Sort one list. For each sum x in the other half, binary-search values nearest $-x$.

Tempting mistake. Greedily give the next weight to the currently lighter side. Weights 8,7,6,5,4 can force a locally plausible decision that misses a perfect balance.

Proof. Every full assignment decomposes uniquely into one assignment per half. For fixed left sum x , the optimal right sum is a sorted value closest to $-x$, found by checking the lower-bound neighbors.

Transfer. Meet in the middle applies when an exponential choice separates into two independently enumerable contributions that combine by a simple operation.

Section 48 — DP optimization

Original problem: Bounded-delay signal

Problem. $dp[i]$ is the minimum cost to reach position i ; the previous position must lie in $[i-W, i-1]$, and transition cost is $base[i] + dp[j]$.

Naive DP. Taking the minimum over W predecessors costs $O(nW)$.

The turn. $base[i]$ does not depend on j , so each transition needs the minimum $dp[j]$ in a sliding index window. Maintain candidate indices in a monotonic deque: discard expired fronts and dominated larger dp values from the back.

Tempting mistake. Use a deque merely because the recurrence has a window. If transition cost also contains an arbitrary term depending on both i and j , ordering candidates by $dp[j]$ alone is invalid.

Invariant. Deque indices are in increasing order, their dp values are nondecreasing, and every removed-back candidate is no better and expires no later than the new candidate.

Transfer. DP optimization begins by isolating the expression minimized over the transition index and identifying its structure.

Section 49 — Convex hull trick and Li Chao trees

Original problem: Quadratic launch calibration

Problem. For increasing positions x_i , compute:

```
dp[i] = setup[i] + min over j<i of (dp[j] + (x_i-x_j)^2)
```

Algebraic turn. Expand the square:

```
dp[j] + x_j^2 - 2*x_i*x_j + x_i^2
```

For fixed j , the part depending on query x_i is a line with slope $-2*x_j$ and intercept $dp[j]+x_j^2$; $x_i^2+setup[i]$ is added afterward.

Tempting mistake. See a quadratic and use ternary search. The minimum is over discrete, changing candidates and need not be unimodal in j .

Structure choice. Increasing x_j gives monotone slopes and increasing queries, permitting a deque hull. Without either monotonicity, use a Li Chao tree over the coordinate domain.

Transfer. CHT is not a memorized recurrence. Expand the transition and ask whether each previous state becomes a linear function of the current query.

Section 50 — Advanced algorithms mixed contest

Original problem: Archive similarity routes

Problem. Documents form nodes. A directed edge exists when one document's signature is a substring of another; edge cost depends quadratically on length difference. Find a cheapest route visiting a required subset of signatures.

Decomposition. This deliberately mixes phases:

```
string algorithm -> discover containment edges
weighted graph   -> compute pairwise required-node distances
bitmask DP       -> choose visit order
```

Tempting mistake. Reach immediately for one advanced structure that seems to explain the whole story. No single tool covers substring detection, path cost, and subset ordering.

Proof contracts. The string phase must produce exactly the modeled edges; shortest paths must use nonnegative costs; the bitmask state must retain the last required node. Validate each phase independently against a tiny direct implementation.

Transfer. Advanced mixed problems are often pipelines. Draw the intermediate object produced by each algorithm before coding any of them.

Section 51 — Invariants in hard problems

Original problem: Paired counter increments

Problem. Arrays A and B contain nonnegative integers. One move chooses two *different* indices and increments both values by one. Decide whether A can be transformed into B .

First invariants. Put $d[i] = B[i] - A[i]$. Every $d[i]$ must be nonnegative, and $\text{sum}(d)$ must be even because every move contributes two increments.

Tempting mistake. Declare those conditions sufficient. The demand vector $[4, 1, 1]$ is nonnegative and has even sum, but the first counter needs four partners while all other counters together offer only two.

The turn. Forget the changing array and view $d[i]$ as the required degree of vertex i in a loop-free multigraph. Each move adds one edge and supplies one unit of degree to each endpoint. This exposes the missing condition: $\max(d) \leq \text{sum}(d) - \max(d)$.

Why the conditions suffice. Repeatedly connect the two vertices with largest remaining demands and decrement both demands. If the old total is S , the old maximum is at most $S/2$; after the move it is at most $S/2 - 1$, which is half the new total. Any untouched demand is no larger either (three demands of at least $S/2$ could not fit in total S). Thus the condition survives until the even total reaches zero. This is the degree-sequence criterion for a loop-free multigraph when parallel edges are allowed.

Transfer. For hard invariant problems, separate necessity from sufficiency. After finding conserved quantities, ask what resource could become stranded, then reinterpret operations as a familiar combinatorial object.

Section 52 — Game theory

Original problem: Mirror-token race

Problem. A position contains two piles. A move removes one token from one pile or one token from both. The player taking the last token wins. Determine winning positions for bounded pile sizes.

Exact model. A state is losing if it has no move to a losing state; it is winning if at least one move reaches a losing state. Fill states by increasing sum of pile sizes.

Tempting mistake. Guess that parity of total tokens decides the game. From $(1,1)$, removing both wins immediately despite even total; parity alone loses the move structure.

The turn. Compute a small table, then inspect patterns only after the winning/losing recurrence is verified. If a periodic pattern emerges, prove that every claimed losing state moves only to winning states and every other state has a move into the losing set.

Transfer. Game theory discovery begins with the recursive definition of winning and losing states, not with guessing a formula from the story.

Section 53 — Advanced combinatorics

Original problem: Necklaces with no adjacent black beads

Problem. Count binary necklaces of length n , considering rotations equivalent, with no adjacent black beads including across the circular boundary.

Why division by n fails. Some strings have smaller rotational periods and therefore fewer than n distinct rotations.

The turn. Burnside's lemma averages the number of valid strings fixed by each rotation. A rotation with cycle length determined by $\gcd(n, k)$ forces a periodic pattern; count valid circular binary patterns on that shorter period, for example by a small DP distinguishing first and last bits.

Tempting mistake. Count linear strings with no adjacent black beads and then divide by n . This ignores both wraparound adjacency and nonuniform orbit sizes.

Proof. Burnside counts pairs (configuration, symmetry fixing it) in two orders, guaranteeing each orbit contributes exactly one after averaging.

Transfer. When objects are considered equal under symmetry, inspect orbit sizes before dividing; nonuniform sizes signal Burnside or Pólya methods.

Section 54 — Geometry

Original problem: Fence gate crossing

Problem. Given an integer-coordinate polygonal fence and many proposed straight gates, determine whether each gate properly crosses any fence edge; touching a vertex must be reported separately.

Primitive. The cross product orientation tells which side of a directed line a point occupies using exact integer arithmetic. Two segments properly cross when endpoint orientations are strictly opposite on both lines.

Tempting mistake. Compare slopes with floating point. Vertical lines need special cases, and near-collinear large coordinates lose precision. Another mistake merges proper crossing with endpoint touching.

The turn. Build one exact segment-relation function returning disjoint categories: separate, proper cross, endpoint touch, collinear overlap. Handle zero orientations with bounding-box checks.

Transfer. Geometry solutions become reliable when composed from a small number of exact predicates with explicit degeneracy contracts.

Section 55 — Randomization and hashing

Original problem: Mirrored log updates

Problem. A string receives point updates and substring palindrome queries.

Model. Maintain polynomial hashes of the string and its reverse in Fenwick trees or segment trees. Normalize queried powers before comparing the forward interval with its mirrored reverse interval.

Tempting mistake. Compare raw hashes from different starting indices; their powers are shifted. Mapping $[l, r)$ to the reverse as $[n-r, n-l)$ is another common off-by-one failure.

Collision contract. Hash equality is probabilistic. Use two independent moduli or a randomized 64-bit base when the risk is acceptable; use an exact algorithm if the problem requires an adversarial guarantee.

Oracle. For small random updates, compare every query with direct character comparison. Hash code is precisely where an independent exact oracle matters.

Transfer. Hashing replaces content by a composable fingerprint; indexing, normalization, and collision policy are part of correctness.

Section 56 — Hard greedy and matroids

Original problem: Maximum-value acyclic sensor links

Problem. Candidate undirected links have positive values. Choose a maximum-total-value subset containing no cycle; connectivity is not required.

Greedy proposal. Sort links by value descending and add a link when DSU says it does not create a cycle.

Why this greedy deserves suspicion. “Take the largest legal item” is not generally correct. It works here because forests form a graphic matroid: every subset of a forest is a forest, and if one forest is larger than another, an edge from the larger can augment the smaller without creating a cycle.

Exchange proof. Compare the greedy forest with an optimum. When greedy takes edge e , adding e to the optimum creates at most one cycle. Remove from that cycle an edge no heavier than e ; feasibility and total value do not worsen. Repeating aligns an optimum with all greedy choices.

Tempting extension. Adding unrelated per-color quotas may destroy the single matroid structure; the same greedy then needs a new proof or may fail.

Transfer. Hard greedy is about identifying the exchange system, not about the sorting direction alone.

Section 57 — Reductions and modeling

Original problem: Dual-channel observatory

Problem. Each observatory must transmit on exactly one of two channels, A or B. Measurements reveal restrictions such as “stations u and v cannot both use A” or “at least one of x and y must use B.” Decide whether all stations can be configured.

The unhelpful view. There are 2^n assignments. Trying the locally less conflicted channel is tempting, but an early harmless-looking choice can force a contradiction far away.

The modeling turn. Give station i a Boolean variable: true means channel A. Each restriction forbids one pair of truth values, so it becomes a clause with two literals. For example, “not both A” is $\text{not } u \text{ or not } v$.

Mechanical reduction. A clause $(p \text{ or } q)$ is equivalent to implications $\text{not } p \rightarrow q$ and $\text{not } q \rightarrow p$. Build the implication graph. The configuration is impossible exactly when some variable and its negation belong to the same strongly connected component.

Proof obligations. Show both directions: every legal configuration satisfies every produced clause, and every satisfying assignment respects the original restrictions. Then prove or invoke the SCC criterion; do not treat the reduction as merely suggestive.

Transfer. Two choices per object plus pairwise forbidden combinations is a strong 2-SAT signal. “Assign objects to compatible partners” suggests matching; “send divisible capacity through alternatives” suggests flow.

Section 58 — Full virtual-contest workflow

Original problem set: The four-terminal mock

Board. A contains a direct frequency count. B asks whether intervals can be ordered without overlap. C is a shortest path with coupons whose exact state is unclear. D asks for a graph construction under parity constraints.

First-pass reasoning. Record evidence, not feelings: A has a complete model and ten-line implementation; B needs an exchange proof; C has an estimated `vertex x coupons-used` state; D has no construction yet. Solve A, spend a short proof box on B, and leave explicit notes before switching.

Tempting mistake. Read all four stories repeatedly while keeping status in your head. That creates false progress and makes a later restart expensive.

When C fails samples. Freeze new edits. Write the state meaning, transition, and final-state rule on paper; build a tiny brute-force oracle without the optimization; compare the first differing state. A wrong answer is evidence about one contract, not a command to rewrite everything.

Upsolve. Classify each loss as modeling, proof, implementation, or contest operation. Re-solve C from a blank state definition and D from the parity invariant. Record the earliest observation that would have redirected the contest attempt.

Transfer. A virtual contest trains decisions under uncertainty. The useful artifact is a timeline of models, evidence, switches, and failure causes—not just the number of accepted problems.

Section 59 — ICPC team strategy

Original simulation: Harbor-control board

Situation. Three teammates share one computer. One problem is ready to code, one needs a graph proof, one has an unexplained wrong answer, and two are unread.

Bad coordination. The strongest coder keeps the keyboard while also thinking about the proof; another teammate silently debugs the same code; the unread problems remain invisible. Individual activity is high, team throughput is low.

Better protocol. Put every problem on a shared board with an owner, status, next uncertainty, and reassessment time. The coder receives a four-line handoff: state, transition or algorithm, correctness reason, and complexity. The debugger gets a reproducible failing case and an explicit suspected contract. The third teammate scans the unread statements.

Keyboard decision. Keyboard time goes to the task closest to a justified submission, not automatically to the hardest problem or strongest teammate. Thinking, proof checking, brute-force construction, and test design continue away from the machine.

Transfer. Team strategy is a scheduling problem with expensive context switches. Visible ownership and small handoff contracts turn parallel thought into accepted submissions.

Section 60 — Final capstone

Original problem: Flooded archive evacuation

Problem. Rooms form a weighted graph. Entering a room may collect one of a few key types; some corridors require a key, and each key type can open at most one locked corridor. Find the minimum evacuation time.

Start with the impossible model. A route is not described by its current room alone: two arrivals can own different unused keys. The exact state is (room, collected-mask, used-mask). This is too large if key types are numerous, which is useful information rather than failure.

Constraint turn. If there are at most eight key types, masks are intended. If there are thousands but only two locked corridors, store the status of those corridors instead. The smaller side of the interaction determines the state.

Pipeline. Define state semantics first, generate legal corridor transitions, then run Dijkstra because weights are nonnegative. A tiny exhaustive search on short walks checks the compressed model; adversarial tests revisit a room with different key histories.

Tempting mistakes. Mark a room visited globally, consume a key when it is collected rather than used, or add Dijkstra before proving that the state keeps all future-relevant information.

Proof shape. Establish state sufficiency, transition equivalence, and then the shortest-path theorem. The capstone lesson is that composing known algorithms still requires a proof at every interface.

Section 61 — ICPC contest readiness

Original drill: Reservation allocator

Problem. Requests arrive with a start time, duration, and reward. At most one request may use a machine at once; maximize reward. The statement is given in the first minute of a timed team drill.

Four-line pre-code contract. State: best reward after considering a prefix of requests ordered by finish time. Transition: skip a request or take it after the last compatible finish. Correctness: the final chosen request partitions an optimal schedule. Complexity: sorting plus predecessor searches gives $O(n \log n)$.

Tempting mistake. “Interval scheduling” triggers earliest-finish greedy, which maximizes count, not reward. The smallest counterexample is two short low-reward requests versus one overlapping high-reward request.

Readiness drill. One teammate derives the contract, another searches for a counterexample to the proposed greedy, and the coder implements only after both agree. After submission, record whether time was lost in recognition, proof, typing, or validation.

Transfer. Readiness is measurable: can the team expose a wrong analogy, produce a complete contract, hand it off, and validate it under a time box?

Section 62 — Codeforces course ladder

Original ladder problem: One-free prefix budget

Problem. An array contains positive costs. For each budget B , find the longest prefix that can be bought if at most one item in that prefix is free.

Gate 1 model. For a prefix ending at r , its minimum payable cost is `prefix_sum[r] - prefix_max[r]`. Positivity makes this value nondecreasing as r grows: adding an item increases the sum and can increase the removable maximum by no more than that item's cost.

Tempting mistake. Binary-search because the answer “looks ordered” without proving the predicate. The monotonicity proof above is the permission to search.

Gate 2 implementation. Precompute prefix sums and prefix maxima, then use the last true position for every query in $O(\log n)$. Test one element, a new maximum at the boundary, equal costs, and a budget of zero.

Upsolve record. Write the missed observation in reusable form: “When one element may be removed from a positive prefix, track the largest contribution; then prove the corrected aggregate is monotone.” Re-implement a day later without notes before advancing.

Transfer. A ladder is not fifty isolated answers. Advancement comes from turning each failed observation into a retrievable pattern and demonstrating that it survives a fresh implementation.

Section 63 — AtCoder difficulty ladder

Original ladder problem: Painted staircase coupons

Problem. Climb n steps using jumps of length one or two. Landing on a red step costs one coupon; blue steps are free. Count ways using exactly k coupons, modulo a given integer.

Constraint calibration. The natural questions are position and coupons used, giving $dp[i][c]$. Before optimizing, compare $n \cdot k$ with the limit. If it fits, the plain state is a feature: it is easy to define and verify.

Transition discovery. Every route to i ends at $i-1$ or $i-2$; add the color cost of i to the coupon coordinate. This last-step partition is both the recurrence and its correctness proof.

Tempting mistake. Reach for matrix exponentiation because n sounds large, or compress the coupon dimension without evidence that future transitions no longer need it. Optimize only the dimension justified by the actual bounds.

Validation. Enumerate all jump sequences for small n , count their red landings, and compare every k . Boundary cases include step zero, an unreachable coupon count, and consecutive red steps.

Transfer. AtCoder-style progress often comes from exact constraint calibration: state the direct DP, calculate it honestly, and optimize only the bottleneck that remains.

A reusable discovery checklist

For a new problem, write these answers before committing to code:

1. What is the smallest exact brute-force model?
2. Which part of that model exceeds the constraints?
3. What information about the past can affect future choices?
4. Which tempting simplification fails on the smallest counterexample?
5. What invariant, monotone predicate, exchange, or last-step partition repairs it?
6. Can every algorithmic object be translated both ways to the statement?
7. What independent oracle can test small instances?
8. Which observation should be saved as a reusable pattern after the solve?

The goal is not to guess the final technique immediately. It is to make each failed model produce evidence that narrows the next one.